Centro Universitário de Excelência
Sistemas de Informação

Ordenando a Fila: Coleções Dinâmicas, Fila FIFO e Algoritmos de Ordenação

Autor:
Caio Josué Ribeiro Bastos
Camilly Vitória Ribeiro Ferreira
Davi Merces da Silva
Filipe Maia Archanjo dos Santos de Santana
Jackson Souza de Souza
Julia Gomes de Souza

Feira de Santana, 2026

Agenda

O objetivo desta apresentação é demonstrar como evoluímos o sistema da MecâniQA, abandonando os arrays estáticos da Fase 1 em favor de coleções dinâmicas, implementando novas entidades de negócio, uma fila de atendimento FIFO e um algoritmo de ordenação construído do zero.

1. Item 1: Modelagem de novas entidades: Cliente, Carro e as regras de bloqueio via Enum

2. Item:
Coleções dinâmicas: por que o ArrayList substitui os arrays estáticos, e o cálculo financeiro sobre eles

3. Item:
Fila de Atendimento (FIFO): a mecânica de chegada e atendimento da oficina

4. Item:
Motor de Ordenação: o algoritmo Bubble Sort implementado do zero, com chave flexível via Enum

5. Item:
Resultados e considerações: testes de QA e próximos passos

Modelagem de Entidades e Regras de Bloqueio

- Novas entidades do negócio: Cliente (código, nome, telefone, email) e Carro (modelo, placa, ano, estilo)

- O estilo do Carro é um Enum restrito a três valores: HATCH, SEDAN e SUV, evitando que o campo aceite qualquer String livre e garantindo segurança de tipo em tempo de compilação

- Um Pedido não guarda uma lista direta de Peça, foi criada a estrutura intermediária ItemPedido, associando qual peça foi pedida e em qual quantidade

- Regra de bloqueio: após o fechamento de uma Ordem de Serviço ou de um Pedido, o sistema bloqueia rigorosamente a adição ou remoção de itens, controlado por um Enum de Status (na OS) e uma flag booleana (no Pedido)

- Um Cliente pode possuir múltiplos carros associados a ele, relação implementada com uma lista dentro da própria classe Cliente

- A OS segue um ciclo de vida rigoroso via Enum de Status: EM_ABERTO → AGUARDANDO_EXECUCAO → EM_EXECUCAO → FINALIZADA



Da Memória Estática à Dinâmica

- Diferente da Fase 1, não sabemos de antemão quantos serviços uma OS ou quantos itens um Pedido terão — por isso, abandonamos os arrays estáticos em favor do ArrayList, classe nativa do pacote java.util

- O ArrayList gerencia a alocação de memória automaticamente, crescendo e diminuindo sob demanda, sem exigir um tamanho máximo definido em tempo de compilação

- O cálculo do valor total (na OS e no Pedido) é feito por um laço for-each, percorrendo o ArrayList e somando o valor de cada item — a mesma lógica de "varrer item por item" da busca linear da Fase 1, só que sem controle manual de índice

- No Pedido, o total é a soma de (preço da peça × quantidade) de cada ItemPedido, subtraído o desconto aplicado



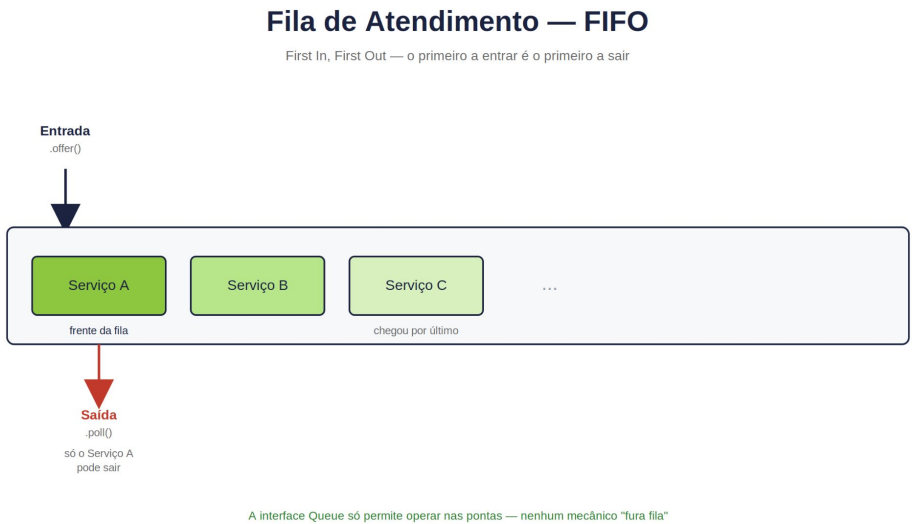
Fila de Atendimento: A Política FIFO

- Quando uma Ordem de Serviço muda para o status AGUARDANDO_EXECUCAO, seus serviços são despachados para uma Fila de Atendimento global

- FIFO significa First In, First Out — o primeiro serviço a entrar na fila é obrigatoriamente o primeiro a ser atendido pelos mecânicos, sem prioridade nem ordem alternativa

- Usamos a interface Queue nativa do Java (implementada por LinkedList), que só expõe operações nas pontas: offer() para adicionar no fim e poll() para remover o item da frente — tornando impossível, por construção da própria API, que um mecânico "fure a fila"

- O abastecimento é feito com um for-each sobre o ArrayList de serviços da OS, preservando a ordem original de chegada ao enfileirar



Motor de Ordenação: Bubble Sort

- Dentre os cinco algoritmos clássicos permitidos (Bubble, Insertion, Selection, Merge, Quick Sort), a equipe escolheu implementar o Bubble Sort do zero

- Justificativa: é o algoritmo mais simples de implementar corretamente, mais fácil de explicar visualmente numa apresentação oral, e sua complexidade O(n²) é plenamente aceitável para o volume de dados do sistema nesta fase

- O método recebe um parâmetro Enum, ChaveOrdenacao (NOME ou CODIGO_IDENTIFICADOR), tornando o critério de ordenação flexível sem duplicar código

- A troca de posição usa uma variável temporária, guardando a referência do elemento antes da sobrescrita — evitando perder o valor original durante o swap



Considerações finais

● Evolução real de arquitetura: a troca de arrays estáticos por ArrayList elimina a limitação de capacidade fixa que a equipe havia defendido como desvantagem no OAT 1

● A Fila FIFO modela fielmente a política de atendimento real de uma oficina, o primeiro carro que chega é o primeiro a ser atendido

● Testes de QA validados: QA-01 (bloqueio de adição pós-fechamento), QA-02 (cálculo de valor total), QA-01 da fila (ordem FIFO respeitada) e QA-06 (ordenação por nome e código): todos passaram

● Próximos passos: persistência dos dados em arquivos CSV, garantindo que o sistema não perca informações ao ser encerrado

Referências

● UNEX — Centro Universitário de Excelência. Guias das Sessões Tutoriais do OAT 2 (Encontros 1, 2 e 3). Feira de Santana, 2026.

● MecâniQA — Automotive Tech. Especificação do Produto Final — OAT 2: Ordenando a Fila. Feira de Santana, 2026.

● Repositório do projeto: github.com/Davibesxxol/mecaniQA-natal